

Contents

tea — plotting + WebAssembly build notes (v1.1 work)	1
What's new	1
1. SVG plotting (native + browser, one renderer)	1
2. Push-mode session API (REPL refactor)	1
3. WebAssembly build (make wasm → web/)	1
Building the WASM bundle	2
WASM regression status	2
Two findings for the author (v1.1 candidates, decisions yours)	2
Update: backend-independent output (portability fix)	2
Toolchain bootstrap without emsdk (v1.6.3 session)	3

tea — plotting + WebAssembly build notes (v1.1 work)

What's new

1. SVG plotting (native + browser, one renderer)

New commands, registered alongside the existing dispatch table:

```
scatter y x [if] [in] [, title() xtitle() ytitle() saving(FILE) noview]
line    y x [if] [in] [, sort title() ... saving() noview]
histogram v [if] [in] [, bins(#) freq title() ... saving() noview]
(hist is an alias)
```

- Self-contained SVG renderer in `src/plot.c / src/plot.h` — no gnuplot, no graphics dependency. 1-2-5 nice-tick axes, gridlines, titles.
- Missing values dropped pairwise; `if/in` handled with the standard expression machinery; string variables rejected with a clear error.
- Native REPL: writes `tea_graph.svg` (or `saving()` target) and opens the OS viewer via `xdg-open/open`. Never auto-opens in do-files, so batch runs and the test suite stay deterministic. `noview` suppresses the viewer.
- Histogram default: density; `freq` for counts; auto bins = `min(ceil(sqrt(N)), 50)`.
- Regression test `40_plot` diffs golden SVG byte-for-byte (suite now 40/40, clean under ASan/UBSan).

2. Push-mode session API (REPL refactor)

`run_stream()`'s loop body is now a state machine, `TeaSession` (`interp.h/interp.c`): `tea_session_new / _feed / _flush / _free`. The native REPL is a thin driver over it; behavior is a faithful port (`#delimit ;, ///` continuation, `{}` block accumulation, comment stripping, EOF-mid-block semantics all preserved — 40/40 regression tests unchanged). This is what makes an event-driven browser front-end possible.

3. WebAssembly build (make wasm → web/)

- `src/wasm_main.c`: browser entry points `tea_web_init`, `tea_web_exec(line)`, `tea_web_run_dofile(path, ext)`, `tea_web_version` (all no-ops in native builds).
- `src/wasm_linalg.c` + `wasm/include/{lapacke,cblas}.h`: a minimal, exact backend for `linalg.h` on top of reference CLAPACK (f2c). This exists because linking the stock LAPACK/CBLAS C wrappers against f2c code hits `wasm-ld`'s strict function-signature checking (f2c subroutines return `int`, the modern headers declare `void`) and traps at

runtime. The shim declares the f2c symbols with their true signatures. Column-major only — every call site in tea is LAPACK_COL_MAJOR.

- readline is stubbed out under `__EMSCRIPTEN__` (interp.c); the xterm.js front-end feeds `tea_session_feed` directly.
- Plots: `plot.c` calls a JS hook (`Module.teaPlot`) instead of a viewer; `web/index.html` renders the SVG inline. WASM and native produce byte-identical SVG (verified on the 40_plot goldens).
- `web/index.html`: xterm.js terminal with history, file upload into MEMFS, download of newly created files, inline plot panel. Static — deployable as-is on GitHub Pages.

Building the WASM bundle

Prebuilt static libs are needed once (paths configurable via `WASM_LIBS/WASM_INC`): - CLAPACK 3.2.1 (github.com/alphacep/clapack) via `emcmake cmake + emmake make lapack` — pre-write `arith.h` (IEEE_8087 / `Arith_Kind_ASA 1` / `Double_Align`) to skip the `run-a-wasm-binary arithchk` step, and note the `s_copy/s_cat` declaration patch below. - GSL (github.com/ampl/gsl) via `emcmake/emmake`. - readstat via `emconfigure ./configure --host=wasm32-unknown-emscripTEN --disable-shared --enable-static`. Then `make wasm` produces `web/tea.js` + `web/tea.wasm` (~800 KB, ~300 KB gzipped).

CLAPACK patch required: 46 SRC files locally declare `int s_copy(...)` / `int s_cat(...)` while `libf2c` defines them `void`. Harmless on native ABIs; a guaranteed trap under `wasm-ld`. A sed one-liner fixes the declarations (see the build log / this session's patch).

WASM regression status

Full suite run inside the module (`web/run_wasm_tests.cjs`, node): - 27/40 byte-identical with the OpenBLAS-generated goldens. - 12 of the 13 differences are pure floating-point noise on *degenerate exact-fit fixtures*: quantities that are mathematically zero (residual SS, SEs of perfectly-fit coefficients) come out as exactly 0 under OpenBLAS but ~1e-16/1e-29 under reference BLAS (or vice versa). Estimates agree to machine precision everywhere. - The 13th (40_plot) is a harness artifact: `shell cat` output under node bypasses the capture hook. The SVGs themselves are byte-identical.

Two findings for the author (v1.1 candidates, decisions yours)

1. **Zero-noise-sensitive output.** Where a variance/SE is mathematically 0, display flips qualitatively with BLAS backend (SE 0 → t 0.00, p 1.000 vs SE 1e-16 → t 1e16, p 0.000; xtreg's "F test that all u_i=0" line appears or vanishes). A relative snap-to-zero tolerance in the display layer (e.g. $RSS < 1e-12 \cdot TSS \Rightarrow 0$) would make output backend-independent. Alternatively, regenerate the affected test fixtures with noise added so they aren't exact fits.
2. **32-bit unsigned long hash constants.** `commands.c`, `eval.c`, `regress.c` use FNV-1a with 1469598103934665603UL; on `wasm32` (long = 32 bits) the constant truncates. Hash tables still work (it's just a different hash), but `uint64_t/ULL` would make the hashing identical across platforms.

Update: backend-independent output (portability fix)

Field testing on a second machine confirmed finding #1 above: several regression fixtures are exact fits, and quantities that are mathematically zero (residual SS, SEs and coefficients of perfectly-fit models, `sigma_u`, hausman V-differences) printed BLAS-backend-dependent noise, failing the suite on any machine other than the one that generated the goldens.

Fixes (display/variance layer, all guarded by “the quantity is a mathematical zero” tests at $1e-12..1e-8$ relative tolerances): - `tea_snap_rss()` in `stats.h`: $RSS < 1e-12 * TSS$ snaps to exactly 0. Applied in `regress` (OLS), `xtreg fe/be/re`. Zero RSS also zeroes the residual vector (keeps robust/cluster sandwich SEs exact) and snaps coefficients below $1e-8$ of the largest (the unique exact solution’s mathematical zeros). `F` on a perfect fit is set to a deterministic $\inf / p$ 0.0000 (the existing golden convention). - `sigma_u`: panel-intercept variance below $1e-24$ of the intercepts’ mean square snaps to 0. - `hausman`: entries of $V_{FE} - V_{RE}$ (and $b_{FE} - b_{RE}$) below relative machine precision snap to 0, so the `sqrt(diag)` column and `chi2` are exact.

Test-side changes where the *checked value itself* is a mathematical zero or an optimizer output (fixtures deliberately exact by design): - 33: intercept-only logit/probit checks converted to tolerance assertions on `_b[_cons]` and `_se[_cons]` (the documented properties), via quietly. - 34: predict’s `e/ue` (FE perfect fit) and `gr` (TS-op exact fit) columns cleaned through `cond(abs(.) < 1e-9, 0, .)` — the tolerance doubles as the assertion since a non-tiny value would survive and fail the diff. - 39: full-precision ARIMA coefficient display replaced by a 0.05 tolerance assertion plus a 4-decimal rounded print. - 20, 34 (first block): exact-fit fixtures de-degenerated with deterministic `mod()` terms (their purpose is weight validation / predict mechanics, not exact-fit display). - shell command: `fork/execl` guarded for **EMSCRIPTEN** (no `fork` in WASM); routed through `system()`, which Emscripten implements.

Validation: 40/40 native (`gcc` + `OpenBLAS`, release and `ASan/UBSan` builds) and 40/40 inside the WASM module (`clang` + reference `CLAPACK/BLAS` + `musl libm`) — byte-identical output across two maximally different numerical backends. The WASM harness (`web/run_wasm_tests.cjs`) runs each test in a subprocess with host `/tmp` mounted via `NODEFS` so shell semantics match the native harness.

Toolchain bootstrap without emsdk (v1.6.3 session)

emsdk’s binary downloads (`storage.googleapis.com`) can be unreachable; the whole build works from distro packages + GitHub sources instead:

1. `apt-get install binaryen clang-15 lld-15 llvm-15 node-acorn`, then `apt-get download emscripten && dpkg -i --force-depends emscripten_*.deb` (the only unmet dep is a nodejs version PIN — any node ≥ 12 works). If node is modern (≥ 18), `npm install acorn` inside `/usr/share/emscripten/tools/` so `acorn-optimizer.js` finds its module.
2. Static libs, all from GitHub, per the recipe above: `alphacep/clapack` (pre-seed `arith.h`, strip the `arithchk` custom command from `F2CLIBS/libf2c/CMakeLists.txt`, sed the 46 `s_copy/s_cat int->void` declarations), `ampl/gsl` (`emcmake`), `WizardMac/ReadStat` (`autoreconf` + `emconfigure`). Stage the five `.a` files in `$WASM_LIBS`.
3. `make wasm` — note `web/tea.js` does NOT depend on `./VERSION` in the Makefile, so after a version bump run `make wasm-clean && make wasm` or the engine reports the old version.
4. Node harness note: `emscripten 3.1.6`’s loader predates node’s global fetch; the test harness is unaffected (it passes `wasmBinary` directly), but ad-hoc node `tea.js` runs `need delete globalThis.fetch` first.